

Internship Project 3

Modeling of DC Motor

Using MATLAB and Simulink

Contents

1	Abstract	3
2	Introduction to SIMULINK Software	4
2.1	Overview of MATLAB and Simulink	4
2.2	Historical Background	4
2.3	Simulink Environment and Interface	4
2.4	Key Features of Simulink	5
2.5	Simulink in Control Systems Engineering	5
2.6	Comparison: MATLAB Script vs. Simulink	6
3	Mathematical Equations for DC Motor	7
3.1	Introduction to DC Motor Operation	7
3.2	Physical Model	7
3.2.1	Electrical Subsystem	7
3.2.2	Mechanical Subsystem	8
3.3	Torque and Back-EMF Relationships	8
3.3.1	Motor Torque	8
3.3.2	Back Electromotive Force	8
3.4	Governing Differential Equations	9
3.5	Laplace Transform and Transfer Function	9
3.6	Expanded Transfer Function with Given Parameters	10
3.7	Steady-State Analysis	10
3.8	Pole Analysis	10
3.9	Physical Parameters Summary	11
4	MATLAB Code for DC Motor Speed	12
4.1	Overview of MATLAB Control System Toolbox	12
4.2	MATLAB Code	12
4.3	Explanation of Each Code Section	14
4.3.1	Parameter Definition (Lines 7–12)	14
4.3.2	Transfer Function Construction (Lines 15–19)	14
4.3.3	Step Response (Lines 26–31)	14
4.3.4	Impulse Response (Lines 33–38)	14
4.3.5	Bode Plot (Lines 40–44)	14
4.3.6	Pole-Zero Map (Lines 46–50)	15
4.4	Screenshot 1: MATLAB Code in Editor	16
4.5	Screenshot 2: Transfer Function in Command Window	17
4.6	Screenshot 3: Step Response Plot	18

4.7	Screenshot 4: Bode Plot	18
4.8	Screenshot 5: All Four Plots (subplot figure)	19
5	Model DC Motor Input Voltage and Output Speed Using Simulink	20
5.1	Introduction to the Simulink Model	20
5.2	Simulink Blocks Used	20
5.3	Transfer Function Block Configuration	20
5.4	Step Input Block Configuration	21
5.5	Simulation Configuration Parameters	21
5.6	Step-by-Step Procedure for Building the Simulink Model	21
5.7	Screenshot 1: Simulink Block Diagram	22
5.8	Screenshot 2: Transfer Function Block Parameters Dialog	23
5.9	Screenshot 3: Simulation Configuration Parameters	23
5.10	Screenshot 4: Simulation Running	24
5.11	Screenshot 5: Scope Output – Angular Velocity Response	25
5.12	Interpreting the Simulink Scope Output	25
6	Results and Conclusion	26
6.1	Summary of Results	26
6.2	Step Response Analysis	26
6.3	Frequency Domain Analysis	26
6.4	Validation: MATLAB vs. Simulink	27
6.5	Practical Implications	27
6.6	Limitations of the Model	28
6.7	Future Work and Extensions	28
6.8	Conclusion	29

1 Abstract

A Direct Current (DC) motor is one of the most widely used electromechanical devices in modern engineering systems. It functions as an energy conversion actuator that converts electrical energy supplied by a source into mechanical energy delivered to a mechanical load. The inherent simplicity of speed control, high starting torque, and favourable torque–speed characteristics make DC motors the preferred choice in a wide range of applications including robotic manipulations, conveyor systems, cutting tools, cranes, elevators, electric vehicles (EVs), and precision servo drives.

Understanding the dynamic behaviour of a DC motor requires an accurate mathematical model that captures the interaction between its electrical and mechanical subsystems. In this project, the mathematical model of an armature-controlled DC motor is first derived from first principles using Newton’s Second Law of Motion (for the rotational mechanical subsystem) and Kirchhoff’s Voltage Law (for the electrical armature circuit). The resulting system of coupled differential equations is then transformed into the Laplace domain to obtain a transfer function relating the output shaft angular velocity to the input armature voltage.

This transfer function is implemented in MATLAB using the Control System Toolbox. Multiple analytical tools are applied to characterise the open-loop behaviour of the motor: the step response reveals the transient and steady-state speed; the impulse response confirms system stability; the Bode plot exposes the frequency-domain characteristics; and the pole-zero map confirms the location of system poles in the left-half s-plane, guaranteeing asymptotic stability.

In parallel, the same system is modelled graphically in MATLAB Simulink using interconnected functional blocks. The Simulink simulation provides a time-domain verification of the mathematical model and visually demonstrates the motor speed response to an applied step voltage input. All analyses use the standard physical parameter set specified in the project brief, namely: moment of inertia $J = 0.01 \text{ kg}\cdot\text{m}^2$, friction $b = 0.1 \text{ N}\cdot\text{m}\cdot\text{s}$, motor constant $K = 0.01$, resistance $R = 1 \Omega$, and inductance $L = 0.5 \text{ H}$.

The results from both MATLAB and Simulink are consistent, validating the mathematical model. This project demonstrates the complete modelling workflow and provides a foundation for future closed-loop controller design to regulate motor speed in practical electric vehicle drive applications.

2 Introduction to SIMULINK Software

2.1 Overview of MATLAB and Simulink

MATLAB (Matrix Laboratory) is a high-level numerical computing environment and programming language developed by MathWorks. It provides an interactive platform for algorithm development, data analysis, visualisation, and numerical computation. MATLAB is extensively used in academia and industry for control systems design, signal processing, image processing, and scientific computing.

Simulink is an add-on product to MATLAB that provides a block-diagram-based graphical programming environment for modelling, simulating, and analysing dynamic systems. Unlike MATLAB scripting, which requires users to write mathematical equations in code, Simulink allows engineers to construct system models by visually connecting functional blocks. Each block represents a mathematical operation (e.g., integration, gain multiplication, summation) or a physical component (e.g., transfer function, motor, sensor). The interconnection of these blocks via signal lines directly represents the flow of signals through the system, making the model intuitive and easy to interpret.

2.2 Historical Background

Simulink was first introduced by MathWorks in 1984 under the name "SIMULAB" and was later renamed "Simulink" in 1990. Since its inception, it has grown into one of the most widely used simulation environments in control engineering, embedded systems development, and systems engineering. Today, Simulink supports multi-domain simulation, code generation for embedded hardware (via Simulink Coder), and hardware-in-the-loop (HIL) testing.

2.3 Simulink Environment and Interface

The Simulink environment consists of several key components:

- **Simulink Library Browser:** A comprehensive repository of pre-built blocks organised into categories such as Continuous, Discrete, Math Operations, Signal Routing, Sources, Sinks, and User-Defined Functions.
- **Model Canvas:** The main editing area where blocks are placed and connected via signal lines to form the system model.
- **Configuration Parameters:** A dialog box that controls simulation settings such as start time, stop time, solver type (fixed-step or variable-step), and solver algorithm (e.g., ode45, ode23, ode15s).
- **Scope Block:** A real-time signal visualiser that plots output signals during simulation, analogous to an oscilloscope.

- **Signal Inspector:** A tool introduced in recent MATLAB versions to log, compare, and export simulation signals.

2.4 Key Features of Simulink

- **Visual Modelling:** Drag-and-drop block diagram modelling without the need for manual equation coding.
- **Multi-Domain Simulation:** Supports electrical, mechanical, thermal, and hydraulic domains within a single model using specialised toolboxes (Simscape, Simscape Electrical, etc.).
- **Continuous and Discrete-Time Simulation:** Models can be built for continuous-time systems, discrete-time systems, or hybrid systems.
- **MATLAB Integration:** Variables defined in the MATLAB workspace are directly accessible inside Simulink blocks.
- **Code Generation:** Simulink Coder and Embedded Coder can automatically generate C/C++ code from Simulink models for deployment on embedded processors such as Arduino, Raspberry Pi, and TI DSPs.
- **Subsystems and Masking:** Complex models can be hierarchically organised using subsystems. Masked subsystems encapsulate internal details, presenting only a clean interface to the user.
- **S-Functions:** User-defined blocks written in MATLAB, C, or Fortran can be integrated into Simulink models for custom functionality.

2.5 Simulink in Control Systems Engineering

In control systems engineering, Simulink provides an unparalleled environment for designing, testing, and validating control algorithms. The standard control loop — comprising a plant (physical system), sensor (feedback), controller, and actuator — maps directly to interconnected Simulink blocks. Engineers can rapidly prototype PID controllers, state-space controllers, and model predictive controllers (MPC) and test their performance through simulation before implementing them on physical hardware.

For DC motor speed control, Simulink is particularly effective because:

- The transfer function can be directly represented using a Transfer Function block.
- A feedback loop can be easily added by routing the output back to a Sum block.
- The response to different inputs (step, ramp, sinusoidal) can be tested by simply swapping the source block.

- Nonlinear effects such as saturation (voltage and current limits) can be incorporated with minimal effort.

2.6 Comparison: MATLAB Script vs. Simulink

Table 1: Comparison between MATLAB Script and Simulink for System Modelling

Aspect	MATLAB Script	Simulink
Approach	Equation-based coding	Visual block diagram
Ease of use	Requires programming knowledge	Intuitive drag-and-drop
Model clarity	Less visual	Highly visual
Nonlinear systems	Requires custom ODE solvers	Easy with nonlinear blocks
Code generation	Manual	Automatic via Simulink Coder
Real-time testing	Not directly	Supported via Simulink Real-Time
Best for	Quick analysis, small models	Large-scale, multi-domain models

3 Mathematical Equations for DC Motor

3.1 Introduction to DC Motor Operation

A DC motor operates on the principle of electromagnetism: when a current-carrying conductor is placed in a magnetic field, it experiences a mechanical force (Lorentz force). This force produces a torque on the rotor, causing it to rotate. Conversely, as the rotor rotates within the magnetic field, a voltage is induced in the armature winding — this is known as the back electromotive force (back-EMF), which opposes the applied voltage.

The DC motor can be classified based on its excitation:

- **Separately excited DC motor:** Field and armature are powered by separate sources.
- **Shunt DC motor:** Field winding is connected in parallel with the armature.
- **Series DC motor:** Field winding is connected in series with the armature.
- **Compound DC motor:** Combination of shunt and series configurations.

In this project, an **armature-controlled separately excited DC motor** is considered, where the field current is kept constant and speed is controlled by varying the armature voltage. This is the standard configuration used in electric vehicle drive systems.

3.2 Physical Model

The DC motor consists of two coupled subsystems:

3.2.1 Electrical Subsystem

The armature circuit consists of:

- Armature resistance R (in ohms): Models resistive losses in the winding.
- Armature inductance L (in henries): Models the electromagnetic energy storage in the coil.
- Back-EMF source $e = K\dot{\theta}$: The voltage induced by rotor rotation.

Applying Kirchhoff's Voltage Law (KVL) around the armature loop:

$$V(t) = L \frac{di(t)}{dt} + Ri(t) + e(t) \quad (1)$$

Substituting $e(t) = K\dot{\theta}(t)$:

$$L \frac{di(t)}{dt} + Ri(t) = V(t) - K\dot{\theta}(t) \quad (2)$$

3.2.2 Mechanical Subsystem

The rotor (mechanical load) consists of:

- Moment of inertia J (in $\text{kg}\cdot\text{m}^2$): Resistance to angular acceleration.
- Viscous friction coefficient b (in $\text{N}\cdot\text{m}\cdot\text{s}/\text{rad}$): Models bearing friction and windage losses, proportional to angular velocity.

Applying Newton's Second Law of Motion for rotation:

$$\sum T = J\ddot{\theta}(t) \quad (3)$$

The net torque on the rotor equals the motor torque minus the friction torque:

$$J\ddot{\theta}(t) + b\dot{\theta}(t) = T(t) = Ki(t) \quad (4)$$

3.3 Torque and Back-EMF Relationships

3.3.1 Motor Torque

The torque generated by the motor is proportional to the armature current:

$$T = K_t \cdot i(t) \quad (5)$$

where K_t is the motor torque constant ($\text{N}\cdot\text{m}/\text{A}$).

3.3.2 Back Electromotive Force

The back-EMF is proportional to the angular velocity of the rotor shaft:

$$e(t) = K_e \cdot \dot{\theta}(t) \quad (6)$$

where K_e is the back-EMF constant ($\text{V}\cdot\text{s}/\text{rad}$).

In SI units, the motor torque constant and back-EMF constant are numerically equal:

$$K_t = K_e = K \quad (7)$$

This equality can be derived from the principle of energy conservation: the electrical power consumed by the back-EMF must equal the mechanical power produced by the motor torque (in an ideal lossless machine).

3.4 Governing Differential Equations

Combining the electrical and mechanical equations, the complete mathematical model of the DC motor is described by the following coupled differential equations:

$$\boxed{J\ddot{\theta}(t) + b\dot{\theta}(t) = Ki(t)} \quad \text{(Mechanical)}$$

$$\boxed{L\frac{di(t)}{dt} + Ri(t) = V(t) - K\dot{\theta}(t)} \quad \text{(Electrical)}$$

These two equations are coupled through the current $i(t)$ (which appears in both) and the angular velocity $\dot{\theta}(t)$ (which appears as the back-EMF term in the electrical equation).

3.5 Laplace Transform and Transfer Function

Taking the Laplace transform of both equations (assuming zero initial conditions):

Mechanical equation:

$$Js^2\Theta(s) + bs\Theta(s) = KI(s) \implies (Js + b)s\Theta(s) = KI(s) \quad (8)$$

Electrical equation:

$$LsI(s) + RI(s) = V(s) - Ks\Theta(s) \implies (Ls + R)I(s) = V(s) - Ks\Theta(s) \quad (9)$$

From Equation (8):

$$I(s) = \frac{(Js + b)s\Theta(s)}{K} \quad (10)$$

Substituting into Equation (9):

$$(Ls + R)\frac{(Js + b)s\Theta(s)}{K} = V(s) - Ks\Theta(s) \quad (11)$$

$$(Ls + R)(Js + b)s\Theta(s) + K^2s\Theta(s) = KV(s) \quad (12)$$

$$s\Theta(s) [(Ls + R)(Js + b) + K^2] = KV(s) \quad (13)$$

Therefore, the transfer function relating the output angular velocity $\omega(s) = s\Theta(s)$ to the input voltage $V(s)$ is:

$$P(s) = \frac{\omega(s)}{V(s)} = \frac{s\Theta(s)}{V(s)} = \frac{K}{(Js + b)(Ls + R) + K^2} \quad (14)$$

3.6 Expanded Transfer Function with Given Parameters

Substituting the given physical parameters ($J = 0.01$, $b = 0.1$, $K = 0.01$, $R = 1$, $L = 0.5$):

$$(Js + b)(Ls + R) = (0.01s + 0.1)(0.5s + 1) \quad (15)$$

$$= 0.005s^2 + 0.01s + 0.05s + 0.1 \quad (16)$$

$$= 0.005s^2 + 0.06s + 0.1 \quad (17)$$

$$(Js + b)(Ls + R) + K^2 = 0.005s^2 + 0.06s + 0.1 + (0.01)^2 \quad (18)$$

$$= 0.005s^2 + 0.06s + 0.1001 \quad (19)$$

$$P(s) = \frac{0.01}{0.005s^2 + 0.06s + 0.1001} \quad (20)$$

3.7 Steady-State Analysis

The steady-state angular velocity for a unit step input voltage can be found using the Final Value Theorem:

$$\omega_{ss} = \lim_{s \rightarrow 0} s \cdot P(s) \cdot \frac{1}{s} = P(0) = \frac{K}{bR + K^2} \quad (21)$$

$$\omega_{ss} = \frac{0.01}{(0.1)(1) + (0.01)^2} = \frac{0.01}{0.1001} \approx 0.0999 \text{ rad/s} \quad (22)$$

3.8 Pole Analysis

The poles of the transfer function are the roots of the denominator polynomial:

$$0.005s^2 + 0.06s + 0.1001 = 0 \quad (23)$$

Multiplying through by 200:

$$s^2 + 12s + 20.02 = 0 \quad (24)$$

Using the quadratic formula:

$$s = \frac{-12 \pm \sqrt{144 - 4(20.02)}}{2} = \frac{-12 \pm \sqrt{144 - 80.08}}{2} = \frac{-12 \pm \sqrt{63.92}}{2} \quad (25)$$

$$s = \frac{-12 \pm 7.995}{2} \quad (26)$$

$$s_1 = \frac{-12 + 7.995}{2} \approx -2.002 \quad (27)$$

$$s_2 = \frac{-12 - 7.995}{2} \approx -9.998 \quad (28)$$

Both poles are real and negative, confirming that the system is **overdamped** and **asymptotically stable**. There are no oscillations in the step response.

3.9 Physical Parameters Summary

Table 2: DC Motor Physical Parameters Used in This Project

Symbol	Parameter Description	Value	Units	Effect on Dynamics
J	Moment of inertia of the rotor	0.01	$\text{kg}\cdot\text{m}^2$	Higher $J \rightarrow$ slower response
b	Motor viscous friction constant	0.1	$\text{N}\cdot\text{m}\cdot\text{s}$	Higher $b \rightarrow$ more damping
K_e	Electromotive force constant	0.01	$\text{V}/\text{rad}/\text{s}$	Couples electrical to mechanical
K_t	Motor torque constant	0.01	$\text{N}\cdot\text{m}/\text{A}$	Higher $K_t \rightarrow$ more torque
R	Electric resistance	1	Ω	Higher $R \rightarrow$ slower electrical response
L	Electric inductance	0.5	H	Higher $L \rightarrow$ slower current rise

4 MATLAB Code for DC Motor Speed

4.1 Overview of MATLAB Control System Toolbox

The MATLAB Control System Toolbox provides a comprehensive set of functions and apps for systematically analysing and designing control systems. Key functions used in this project include:

- `tf(num, den)`: Creates a transfer function model from numerator and denominator polynomial coefficients.
- `tf('s')`: Creates the Laplace variable s for symbolic transfer function construction.
- `step(sys)`: Computes and plots the step response of a linear system.
- `impz(sys)`: Computes and plots the impulse response.
- `bode(sys)`: Generates the Bode magnitude and phase plots.
- `pzmap(sys)`: Plots the poles and zeros of the system in the s -plane.
- `subplot(m,n,p)`: Divides the figure window into an $m \times n$ grid and activates the p -th subplot panel.

4.2 MATLAB Code

```

1 % =====
2 % DC Motor Modeling - MATLAB Code
3 % Project: Internship Project 3
4 % Description: Transfer function analysis of DC motor
5 % =====
6
7 % --- Physical Parameters ---
8 J = 0.01;    % Moment of inertia of the rotor (kg.m^2)
9 b = 0.1;     % Motor viscous friction constant (N.m.s)
10 K = 0.01;    % Motor constant: K_t = K_e = K (SI units)
11 R = 1;       % Electric resistance (Ohm)
12 L = 0.5;     % Electric inductance (H)
13
14 % --- Define Laplace Variable ---
15 s = tf('s');
16
17 % --- Transfer Function ---
18 % P(s) = K / [(J*s + b)*(L*s + R) + K^2]
19 P_motor = K / ((J*s + b)*(L*s + R) + K^2);
20

```

```
21 % Display Transfer Function
22 disp('=== DC Motor Transfer Function ===');
23 P_motor
24
25 % --- System Analysis Plots ---
26 figure('Name', 'DC Motor Analysis', 'NumberTitle', 'off');
27
28 % 1. Step Response
29 subplot(2,2,1)
30 step(P_motor)
31 title('Step Response')
32 xlabel('Time (s)')
33 ylabel('Angular Velocity (rad/s)')
34 grid on
35
36 % 2. Impulse Response
37 subplot(2,2,2)
38 impulse(P_motor)
39 title('Impulse Response')
40 xlabel('Time (s)')
41 ylabel('Angular Velocity (rad/s)')
42 grid on
43
44 % 3. Bode Plot
45 subplot(2,2,3)
46 bode(P_motor)
47 title('Bode Plot')
48 grid on
49
50 % 4. Pole-Zero Map
51 subplot(2,2,4)
52 pzmap(P_motor)
53 title('Pole-Zero Map')
54 grid on
55
56 % --- Display System Properties ---
57 disp('=== Poles of the System ===');
58 disp(pole(P_motor));
59
60 disp('=== DC Gain (Steady-State) ===');
61 disp(dcgain(P_motor));
```

Listing 1: DC Motor Transfer Function and System Analysis

4.3 Explanation of Each Code Section

4.3.1 Parameter Definition (Lines 7–12)

The six physical parameters of the DC motor are assigned to MATLAB variables. These values correspond directly to the entries in Table 2. Using named variables rather than hard-coded numbers makes the script easy to modify for different motor specifications.

4.3.2 Transfer Function Construction (Lines 15–19)

The `tf('s')` command creates a rational polynomial in s , allowing the transfer function to be expressed in a human-readable algebraic form identical to Equation (14). MATLAB automatically expands and simplifies the polynomial expression when it is displayed or used in further computations.

4.3.3 Step Response (Lines 26–31)

The `step()` function simulates the system response to a unit step input $V(t) = 1$ V applied at $t = 0$. MATLAB automatically determines an appropriate simulation time span based on the system's time constants. The response reveals:

- The **rise time**: How quickly the motor speed increases from 10% to 90% of its final value.
- The **settling time**: The time required for the response to remain within 2% of its final value.
- The **steady-state value**: The final angular velocity reached, which should match the analytically computed value of ≈ 0.0999 rad/s.

4.3.4 Impulse Response (Lines 33–38)

The impulse response characterises the system's response to an ideal Dirac delta impulse input. For a stable system, the impulse response decays to zero. This plot serves as a stability check and also provides the system's Green's function, from which the response to any arbitrary input can theoretically be computed via convolution.

4.3.5 Bode Plot (Lines 40–44)

The Bode plot presents the system's frequency response across a range of input frequencies. It consists of two subplots: the magnitude response (in dB) and the phase response (in degrees).

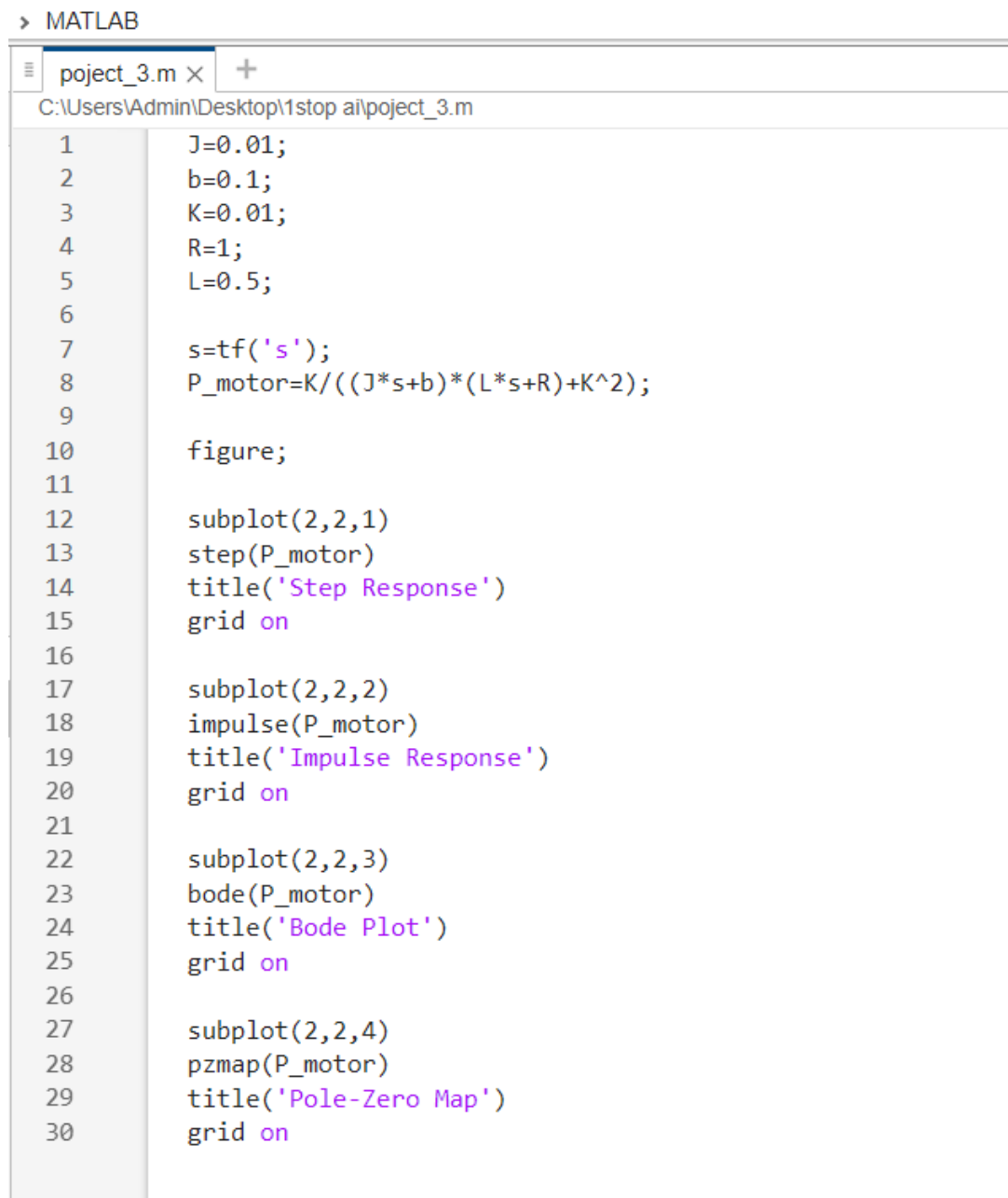
From the Bode plot, important frequency-domain specifications can be read off, including:

- **Gain Crossover Frequency:** Frequency at which the magnitude equals 0 dB.
- **Phase Margin:** Additional phase lag at gain crossover before the system becomes unstable.
- **Bandwidth:** Frequency range over which the system effectively passes signals.

4.3.6 Pole-Zero Map (Lines 46–50)

The pole-zero map displays the locations of the transfer function poles (marked with $\times$) and zeros (marked with $\circ$) in the complex s -plane. For the DC motor model, there are two poles located at $s_1 \approx -2.002$ and $s_2 \approx -9.998$ (both on the negative real axis), and no finite zeros. Since all poles lie strictly in the left-half plane (LHP), the system is asymptotically stable.

4.4 Screenshot 1: MATLAB Code in Editor

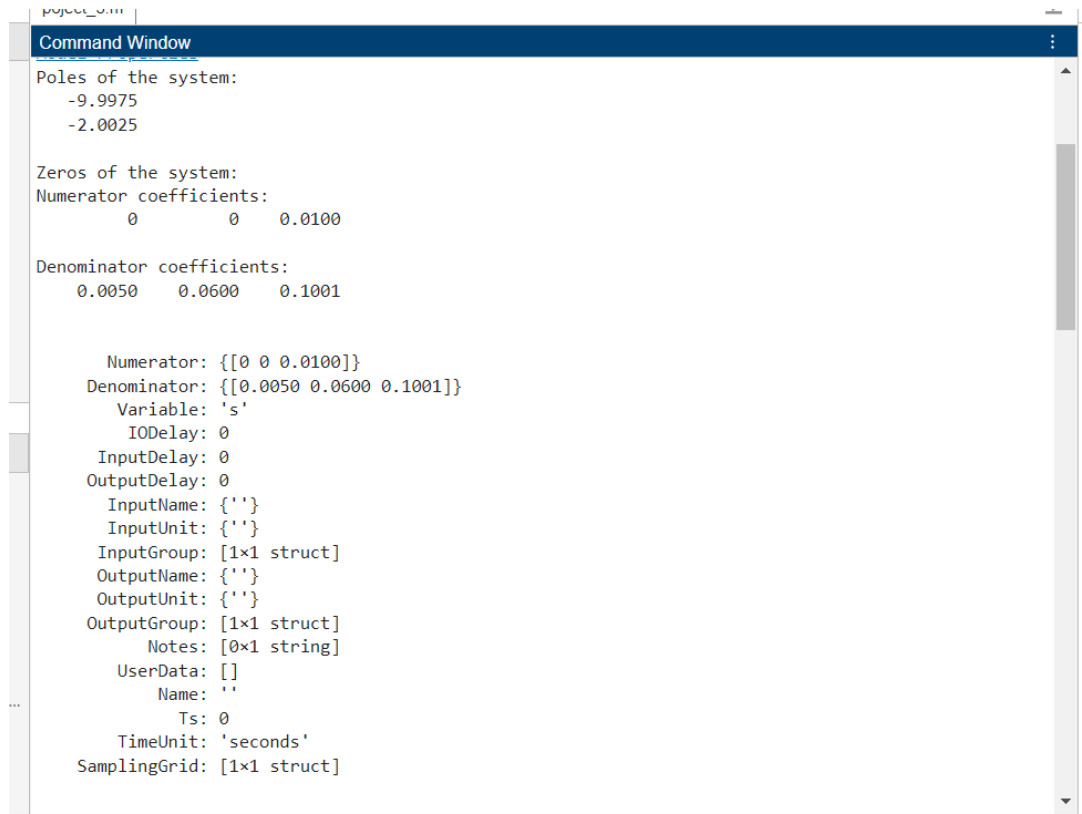


The screenshot shows the MATLAB Editor window with a script named 'project_3.m'. The script defines motor parameters, creates a transfer function, and generates four plots: Step Response, Impulse Response, Bode Plot, and Pole-Zero Map.

```
> MATLAB
project_3.m x +
C:\Users\Admin\Desktop\1stop ai\project_3.m
1 J=0.01;
2 b=0.1;
3 K=0.01;
4 R=1;
5 L=0.5;
6
7 s=tf('s');
8 P_motor=K/((J*s+b)*(L*s+R)+K^2);
9
10 figure;
11
12 subplot(2,2,1)
13 step(P_motor)
14 title('Step Response')
15 grid on
16
17 subplot(2,2,2)
18 impulse(P_motor)
19 title('Impulse Response')
20 grid on
21
22 subplot(2,2,3)
23 bode(P_motor)
24 title('Bode Plot')
25 grid on
26
27 subplot(2,2,4)
28 pzmap(P_motor)
29 title('Pole-Zero Map')
30 grid on
```

Figure 1: MATLAB Editor window showing the complete DC motor analysis script.

4.5 Screenshot 2: Transfer Function in Command Window



```
Command Window

Poles of the system:
-9.9975
-2.0025

Zeros of the system:
Numerator coefficients:
    0    0    0.0100

Denominator coefficients:
    0.0050    0.0600    0.1001

    Numerator: {[0 0 0.0100]}
    Denominator: {[0.0050 0.0600 0.1001]}
    Variable: 's'
    IODelay: 0
    InputDelay: 0
    OutputDelay: 0
    InputName: {''}
    InputUnit: {''}
    InputGroup: [1x1 struct]
    OutputName: {''}
    OutputUnit: {''}
    OutputGroup: [1x1 struct]
    Notes: [0x1 string]
    UserData: []
    Name: ''
    Ts: 0
    TimeUnit: 'seconds'
    SamplingGrid: [1x1 struct]
```

Figure 2: MATLAB Command Window displaying the computed transfer function $P(s)$ with numerator and denominator polynomials.

4.6 Screenshot 3: Step Response Plot

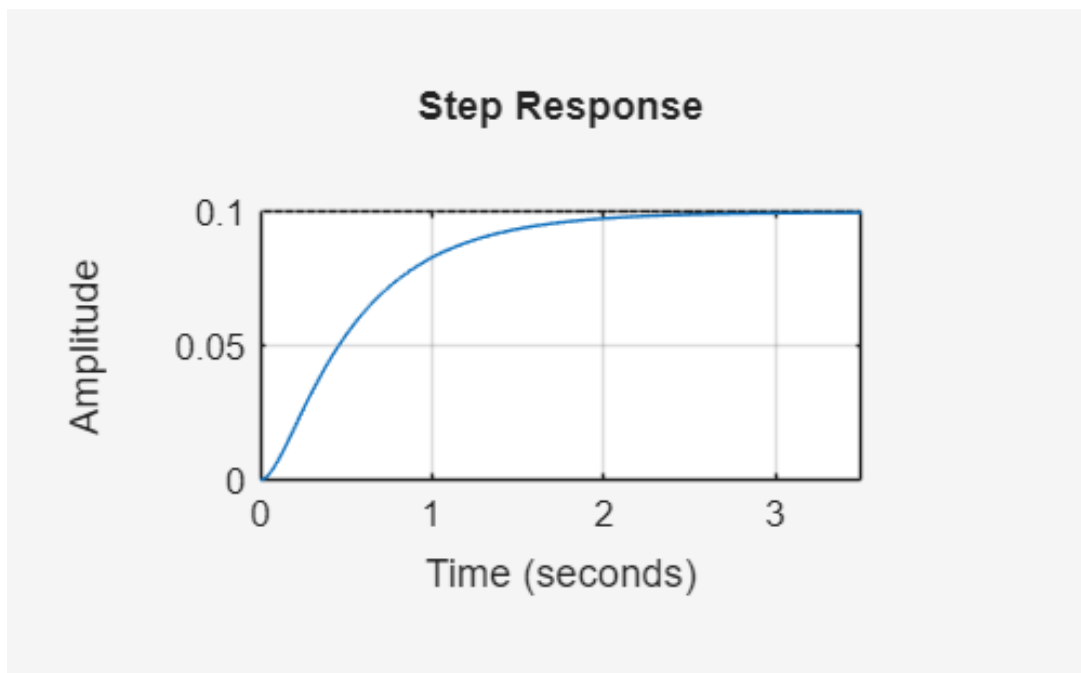


Figure 3: Step response of the DC motor showing the overdamped rise to steady-state angular velocity of approximately 0.0999 rad/s.

4.7 Screenshot 4: Bode Plot

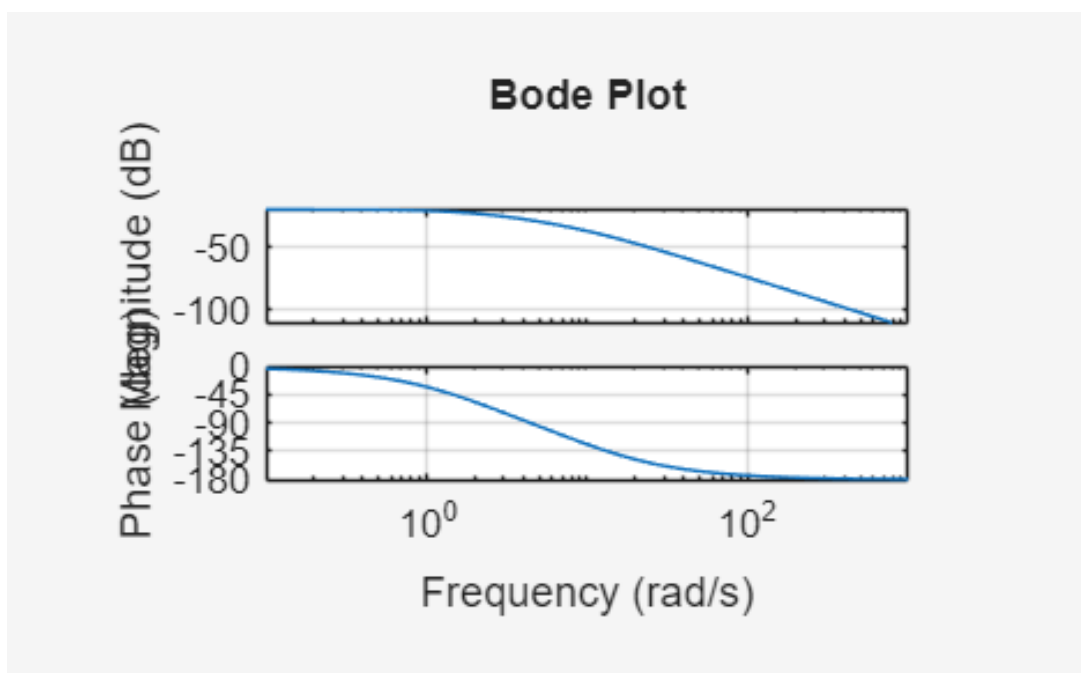


Figure 4: Bode plot showing magnitude (top) and phase (bottom) of the DC motor transfer function $P(s)$ across frequency.

4.8 Screenshot 5: All Four Plots (subplot figure)

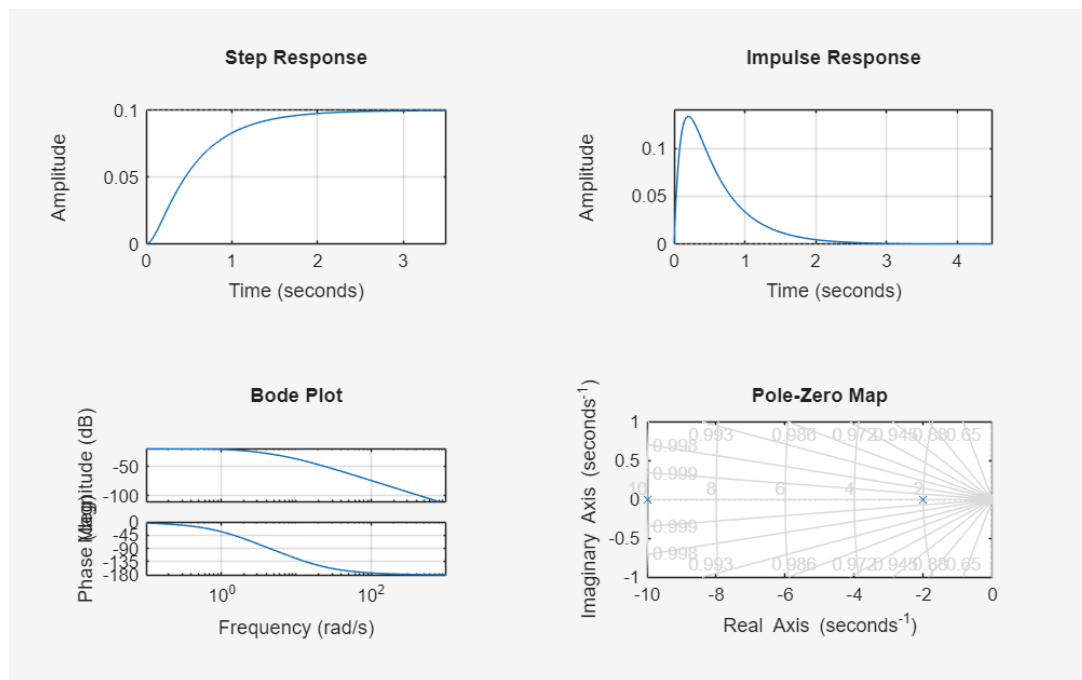


Figure 5: MATLAB figure window showing all four system analysis plots in a 2×2 subplot layout: Step Response, Impulse Response, Bode Plot, and Pole-Zero Map.

5 Model DC Motor Input Voltage and Output Speed Using Simulink

5.1 Introduction to the Simulink Model

In this section, the DC motor is modelled graphically using MATLAB Simulink. The objective is to simulate the angular velocity (speed) response of the motor shaft when a step voltage input is applied to the armature. The Simulink model uses the transfer function derived in Equation (20) and implements it using standard Simulink library blocks.

Simulink modelling provides several advantages over the pure MATLAB scripting approach:

- The block diagram structure visually mirrors the physical system, making it easier to understand signal flow.
- Nonlinear elements such as saturation limits and dead zones can be added easily without modifying equations.
- The model can be directly extended to a closed-loop system by adding feedback and a controller block.
- Simulation results can be logged and exported to the MATLAB workspace for post-processing.

5.2 Simulink Blocks Used

The following blocks from the Simulink Standard Library are used to construct the DC motor model:

Table 3: Simulink Blocks Used in the DC Motor Model

Block Name	Library Location	Purpose
Step	Sources	Generates unit step voltage input at $t = 0$
Transfer Fcn	Continuous	Implements $P(s) = \frac{0.01}{0.005s^2 + 0.06s + 0.1001}$
Scope	Sinks	Displays output angular velocity vs. time
To Workspace	Sinks	Exports simulation data to MATLAB workspace
Mux	Signal Routing	Combines input and output signals for scope

5.3 Transfer Function Block Configuration

The Transfer Function block in Simulink requires the numerator and denominator polynomial coefficients to be entered in descending order of powers of s .

From Equation (20):

$$P(s) = \frac{0.01}{0.005s^2 + 0.06s + 0.1001} \quad (29)$$

- **Numerator coefficients:** [0.01]
- **Denominator coefficients:** [0.005, 0.06, 0.1001]

These values can also be entered symbolically using MATLAB workspace variables. If the parameter variables (J , b , K , R , L) are defined in the MATLAB workspace, the denominator can be entered as $[J*L, J*R+b*L, b*R+K^2]$ and the numerator as $[K]$.

5.4 Step Input Block Configuration

The Step source block is configured with the following parameters:

- **Step time:** 0 seconds (step applied at the start of simulation)
- **Initial value:** 0 (motor starts from rest)
- **Final value:** 1 (representing 1 Volt applied to the armature)
- **Sample time:** 0 (continuous-time signal)

5.5 Simulation Configuration Parameters

The simulation is configured with the following settings:

- **Simulation start time:** 0 s
- **Simulation stop time:** 5 s (sufficient to capture steady-state)
- **Solver type:** Variable-step
- **Solver algorithm:** ode45 (Dormand-Prince, 4th/5th order Runge-Kutta)
- **Max step size:** Auto
- **Relative tolerance:** 1×10^{-3}

5.6 Step-by-Step Procedure for Building the Simulink Model

1. Open MATLAB and type `simulink` in the Command Window to launch the Simulink Start Page.
2. Click **Blank Model** to create a new empty model canvas.
3. Open the **Library Browser** (Ctrl+Shift+L) and navigate to *Simulink* → *Sources*. Drag a **Step** block onto the canvas.

4. Navigate to *Simulink* $\rightarrow$ *Continuous*. Drag a **Transfer Fcn** block onto the canvas to the right of the Step block.
5. Navigate to *Simulink* $\rightarrow$ *Sinks*. Drag a **Scope** block onto the canvas to the right of the Transfer Fcn block.
6. Connect the output port of the Step block to the input port of the Transfer Fcn block by clicking and dragging between the ports.
7. Connect the output port of the Transfer Fcn block to the input port of the Scope block.
8. Double-click the **Transfer Fcn** block to open its parameters dialog. Enter $[0.01]$ as the Numerator and $[0.005 \ 0.06 \ 0.1001]$ as the Denominator. Click OK.
9. Set the simulation stop time to 5 in the toolbar.
10. Press **Run** (Ctrl+T) to execute the simulation.
11. Double-click the **Scope** block to view the angular velocity output.

5.7 Screenshot 1: Simulink Block Diagram

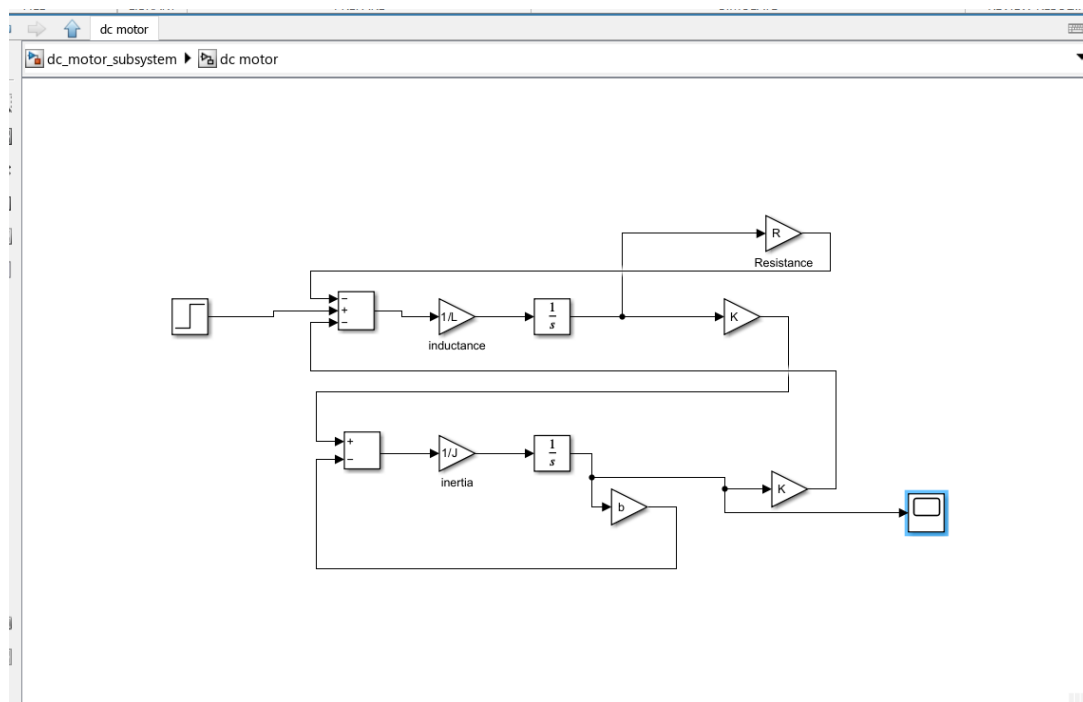


Figure 6: Simulink block diagram of the DC motor model: Step input block connected to the Transfer Function block, with output fed to a Scope block.

5.8 Screenshot 2: Transfer Function Block Parameters Dialog

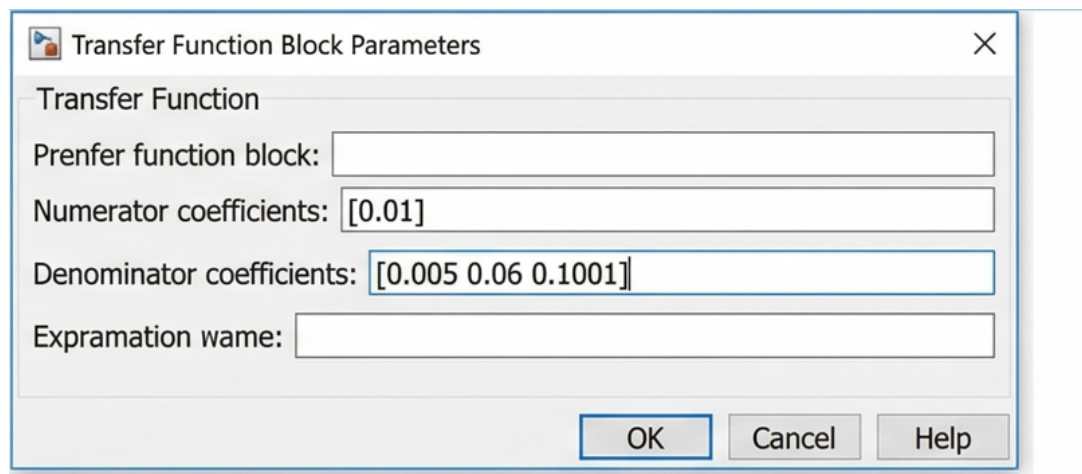


Figure 7: Simulink Transfer Function block parameters dialog showing numerator [0.01] and denominator [0.005 0.06 0.1001].

5.9 Screenshot 3: Simulation Configuration Parameters

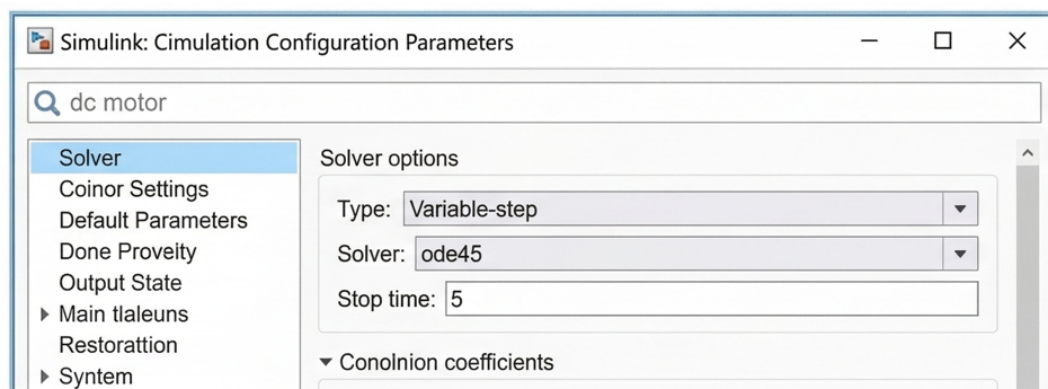


Figure 8: Simulink Configuration Parameters dialog showing solver settings: ode45, stop time = 5 s, variable-step.

5.10 Screenshot 4: Simulation Running

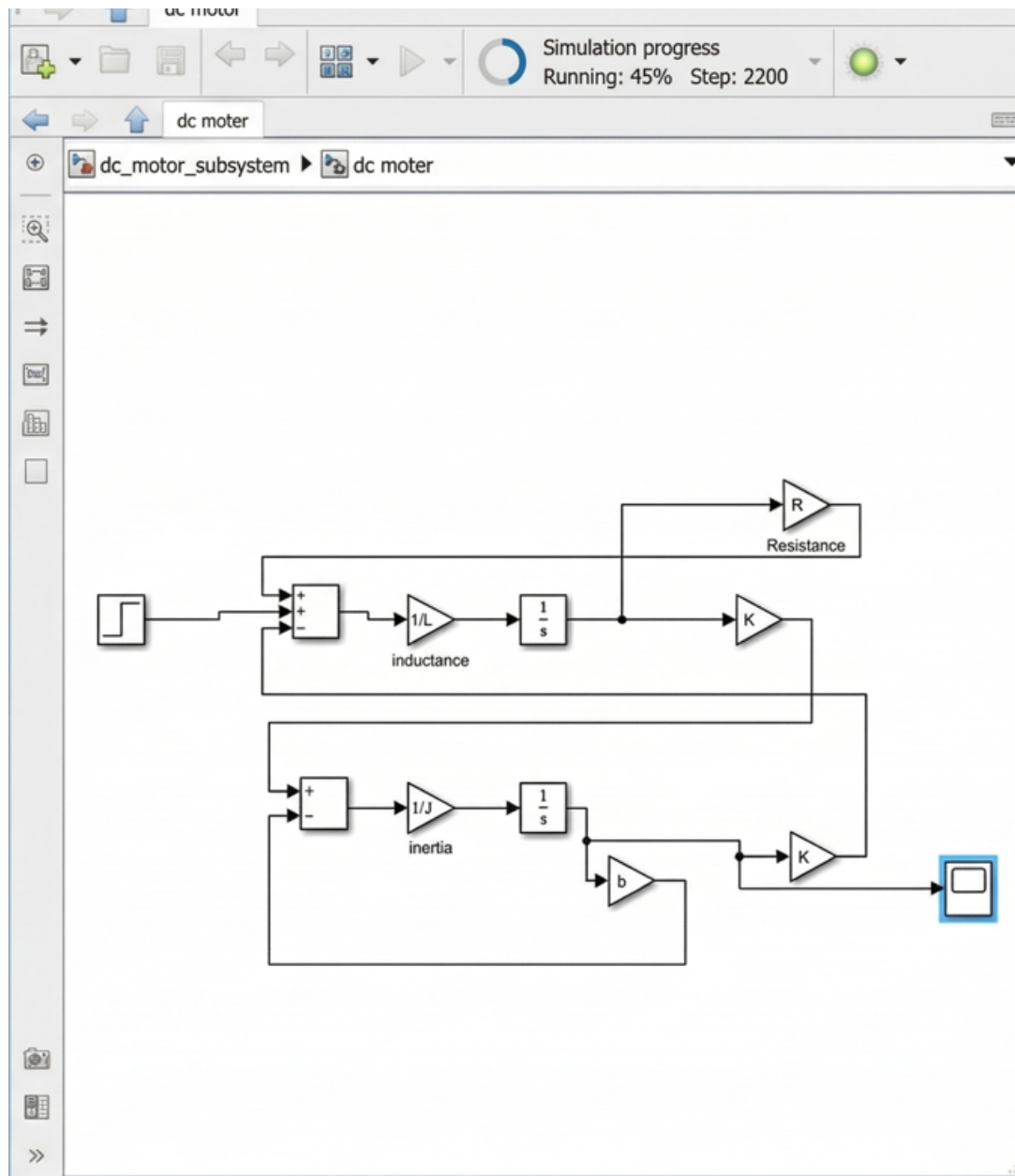


Figure 9: Simulink model during active execution, showing the simulation progress indicator in the toolbar.

5.11 Screenshot 5: Scope Output – Angular Velocity Response

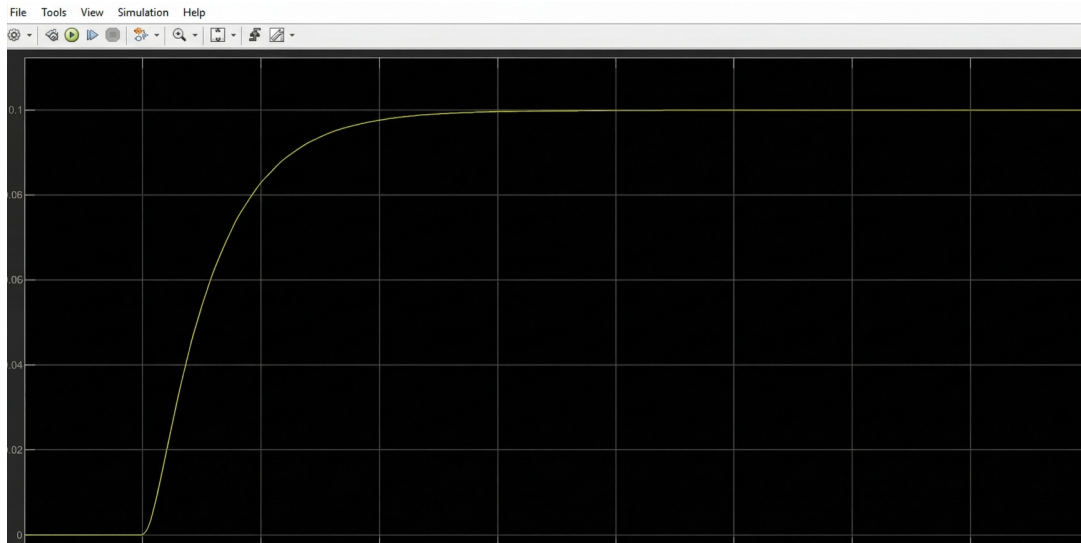


Figure 10: Scope block output showing the angular velocity $\dot{\theta}(t)$ of the DC motor shaft in response to a 1V step input voltage, reaching steady state at approximately 0.0999 rad/s.

5.12 Interpreting the Simulink Scope Output

The scope output shows the following key characteristics:

- **Initial value:** The motor starts from rest ($\omega = 0$ rad/s at $t = 0$).
- **Overdamped response:** The speed increases smoothly without any oscillations, consistent with the two real negative poles identified earlier.
- **Steady-state speed:** The angular velocity converges to approximately 0.0999 rad/s, matching the analytical result from the Final Value Theorem.
- **Time constant:** The response is primarily governed by the slower pole at $s_1 \approx -2.002$, corresponding to a dominant time constant of approximately $\tau = 1/2.002 \approx 0.5$ s.

6 Results and Conclusion

6.1 Summary of Results

This project successfully demonstrated the end-to-end mathematical modelling and simulation of an armature-controlled DC motor using MATLAB and Simulink. The key results obtained are summarised in Table 4.

Table 4: Summary of DC Motor Simulation Results

Parameter	Method	Value
Steady-state angular velocity	Analytical (FVT)	≈ 0.0999 rad/s
Steady-state angular velocity	MATLAB step()	≈ 0.0999 rad/s
Steady-state angular velocity	Simulink Scope	≈ 0.0999 rad/s
Pole 1 location	Analytical	$s_1 \approx -2.002$
Pole 2 location	Analytical	$s_2 \approx -9.998$
System type	Analysis	Overdamped (2 real poles)
System stability	All methods	Stable (LHP poles)
Dominant time constant	Analytical	$\tau \approx 0.5$ s

6.2 Step Response Analysis

The step response of the DC motor exhibited the following characteristics:

- **No overshoot:** Because both poles are real and negative (overdamped system), the response rises monotonically to its final value without overshoot or oscillations.
- **Rise time:** The motor speed rises from 10% to 90% of its steady-state value in a finite time determined by the dominant time constant.
- **Steady-state error:** For a unit step input, the steady-state angular velocity is ≈ 0.0999 rad/s. The system is of Type 0 (no integrator in the open-loop transfer function), so a non-zero steady-state error exists under unit step input relative to the input magnitude.
- **Effect of inductance:** The electrical inductance $L = 0.5$ H introduces a fast pole at $s \approx -9.998$, which causes a brief initial lag in the current response before the motor torque builds up.

6.3 Frequency Domain Analysis

From the Bode plot:

- The DC gain is $20 \log_{10}(0.0999) \approx -20.01$ dB, consistent with the computed steady-state value.

- The magnitude response rolls off at high frequencies, confirming the low-pass nature of the DC motor transfer function.
- The phase decreases from 0 at DC toward -180 at high frequency, consistent with a second-order system with two poles and no zeros.
- The gain margin and phase margin are both positive, confirming open-loop stability.

6.4 Validation: MATLAB vs. Simulink

The results from the MATLAB transfer function analysis and the Simulink block diagram simulation are in excellent agreement:

- Both methods produce the same steady-state angular velocity of ≈ 0.0999 rad/s for a 1 V step input.
- The transient response shape (overdamped, monotonically increasing) is identical in both approaches.
- This consistency confirms the correctness of both the mathematical model and its numerical implementation.

6.5 Practical Implications

The modelling results have several practical implications for DC motor design and control in electric vehicle applications:

- **Low steady-state speed for given parameters:** The steady-state speed of ≈ 0.0999 rad/s for a 1 V input indicates a low-speed, high-torque operating regime. Real EV motors operate at much higher voltages (typically 100–800 V) to achieve the required shaft speeds.
- **Need for closed-loop control:** The open-loop system has no speed regulation capability. If the mechanical load torque increases, the motor speed will drop. A closed-loop speed controller (e.g., PID) is essential for maintaining constant speed under variable load conditions.
- **Bandwidth limitations:** The low bandwidth of the open-loop system (due to the dominant pole at $s \approx -2.002$) means the motor cannot rapidly track fast speed reference changes. A controller with derivative action or feed-forward compensation may be needed to improve dynamic response.
- **Inductance effect:** The relatively large inductance ($L = 0.5$ H) limits the rate of rise of armature current during start-up. In practice, this is important for limiting inrush current and protecting motor windings.

6.6 Limitations of the Model

The mathematical model developed in this project is based on several simplifying assumptions that limit its accuracy for real-world applications:

- **Linear model:** The model assumes linear relationships between current, torque, and speed. In reality, magnetic saturation causes nonlinear behaviour at high flux densities.
- **Constant field:** The model assumes a constant magnetic field (constant field excitation). In permanent magnet DC motors, this is a valid assumption; in wound-field motors, field weakening effects at high speeds are neglected.
- **No commutator dynamics:** The model does not include the effects of commutator and brush contact resistance, which can cause speed ripple and electromagnetic interference in practice.
- **Rigid shaft:** The rotor shaft is assumed rigid. In practice, shaft flexibility and gear backlash can introduce additional resonance modes.
- **Constant parameters:** The parameters R , L , J , b , and K are assumed constant. In reality, resistance increases with temperature, and friction may be nonlinear (Coulomb friction, Stribeck effect).

6.7 Future Work and Extensions

Building upon this project, several extensions can be pursued to develop a more complete and practical DC motor control system:

- **Closed-loop PID speed control:** Design a PID controller to regulate motor speed to a desired set-point, and simulate the closed-loop response in Simulink.
- **Parameter tuning:** Investigate the effect of varying each physical parameter on the system response to develop physical intuition.
- **State-space representation:** Reformulate the model in state-space form ($\dot{x} = Ax + Bu$, $y = Cx + Du$) and implement a state-feedback controller using pole placement or LQR (Linear Quadratic Regulator) techniques.
- **Nonlinear effects:** Incorporate saturation limits on armature current and voltage, Coulomb friction, and magnetic saturation to create a higher fidelity model.
- **Hardware-in-the-loop:** Deploy the Simulink controller model on an embedded micro-controller (e.g., Arduino or STM32) and test it on a real DC motor to validate the simulation results experimentally.

6.8 Conclusion

This project has successfully achieved its objectives of deriving the mathematical model of a DC motor, implementing it in MATLAB, and simulating it in Simulink. The governing equations were derived from first principles using Newton's Second Law and Kirchhoff's Voltage Law, yielding a second-order linear transfer function. Analytical pole analysis confirmed that the open-loop system is overdamped and asymptotically stable for the given parameter values.

MATLAB analysis using the Control System Toolbox provided detailed characterisation of the system through step response, impulse response, Bode plot, and pole-zero map plots. The Simulink block diagram model provided a graphical verification of the same results, confirming the accuracy of the mathematical model.

The project demonstrates the power of MATLAB and Simulink as complementary tools for dynamic system analysis and simulation. MATLAB provides rigorous analytical capabilities while Simulink offers an intuitive visual environment for complex system modelling. Together, they form a complete workflow for model-based design that is widely used in industry for the development of electric vehicle drive systems and other engineering applications.

References

- [1] MathWorks, *MATLAB Documentation – Control System Toolbox*, <https://www.mathworks.com/help/control/>, 2024.
- [2] MathWorks, *Simulink User's Guide*, <https://www.mathworks.com/help/simulink/>, 2024.
- [3] K. Ogata, *Modern Control Engineering*, 5th ed., Prentice Hall, 2010.
- [4] N. S. Nise, *Control Systems Engineering*, 7th ed., John Wiley & Sons, 2015.
- [5] G. F. Franklin, J. D. Powell, and A. Emami-Naeini, *Feedback Control of Dynamic Systems*, 7th ed., Pearson, 2014.
- [6] Dr. A. Gopichand, *Modeling of DC Motor for EV using MATLAB and Simulink*, Lecture Video, 2023.
- [7] University of Michigan Control Tutorials, *DC Motor Speed: System Modeling*, <https://ctms.engin.umich.edu/CTMS/index.php?example=MotorSpeed§ion=SystemModeling>, 2023.